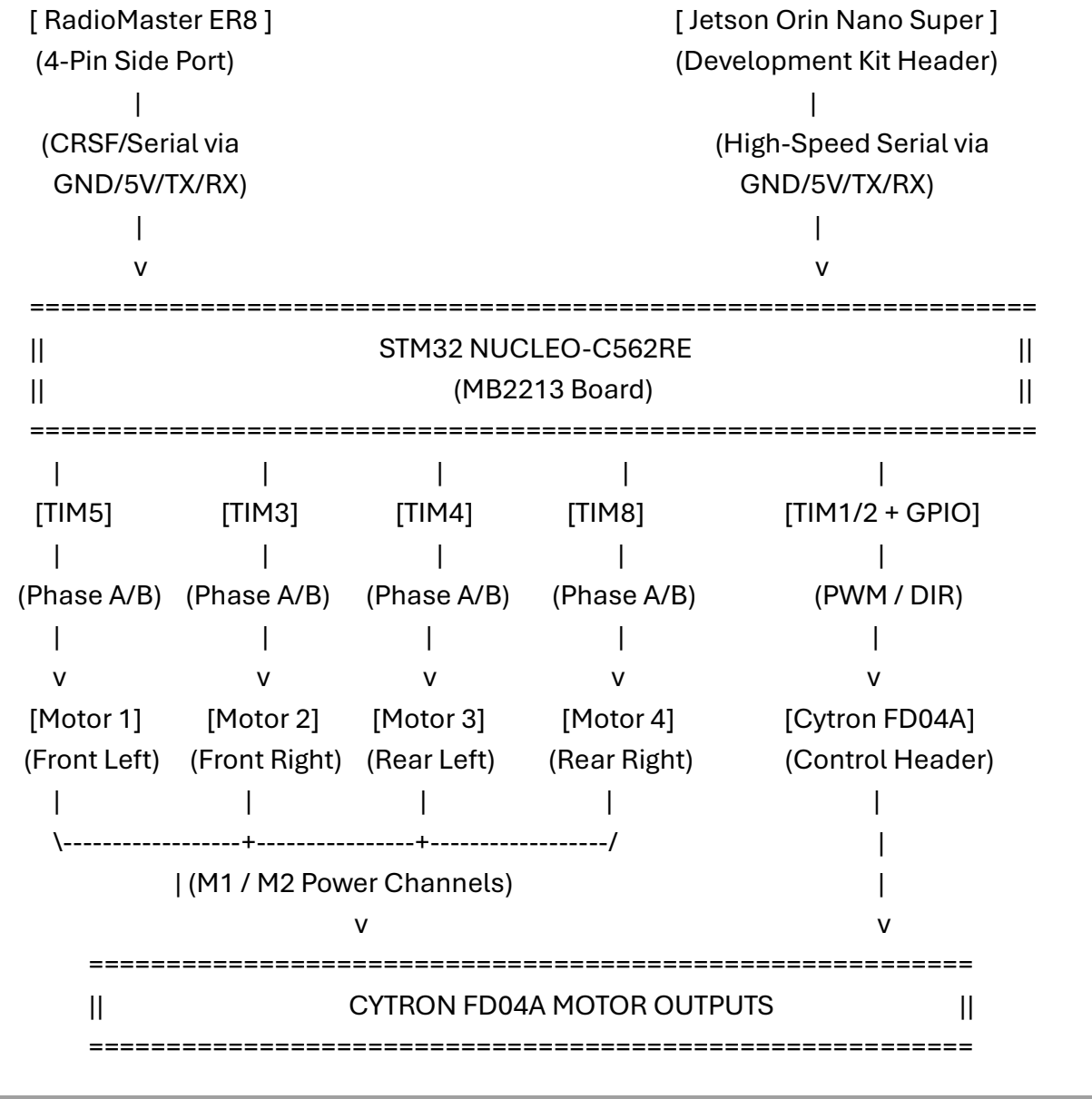


This is a master integration plan for your **STM32 Nucleo-C562RE (MB2213 board)**, **RadioMaster ER8 ExpressLRS Receiver**, **Cytron FD04A (4-Channel Motor Driver)**, and four **JG37-520 Encoder Motors**. It details every connection, pin name, physical location, and essential hardware precaution to ensure accurate assembly.

Hardware Topology Mapping



Module 1: [RadioMaster ER8](#) to STM32 Nucleo-C562RE

The RadioMaster ER8 decodes instructions from your TX16S transmitter. It sends all channels as a single digital stream over a dedicated serial connection.

Target Port on ER8

Locate the **4-pin JST-GH vertical receptacle** on the side of the receiver casing (labeled **CRSF** or **Serial**). Do not confuse this with the 3-pin row headers used for traditional PWM servos. [\[1\]](#)

Connections & Locations

Wire # [1]	ER8 Pin Label	Wire Color (Standard)	Nucleo Pin Name	Physical Pin Location (ST Morpho Connector)
1	- (GND)	Black	GND	CN10 Pin 9 (or CN7 Pin 20)
2	+ (5V)	Red	+5V	CN7 Pin 18
3	RX	Yellow/White	PA9 (TX1)	CN10 Pin 21
4	TX	Green/Blue	PA10 (RX1)	CN10 Pin 33

Module Warnings & Precautions

- **The Transmit/Receive Crossover Rule:** Ensure ER8 **TX** links to Nucleo **PA10 (RX)** and ER8 **RX** links to Nucleo **PA9 (TX)**. Wiring TX-to-TX or RX-to-RX will block communication.
 - **Overvoltage Destructive Boundary:** The 4-pin side connector operates strictly at **5V**. Do not connect a raw LiPo battery (such as a 3S or 4S battery) to this port. The EXT-V pin on the other end of the receiver is the only port designed to read battery voltages up to 35V.
-

Module 2: Cytron FD04A Control Interfacing to STM32 Nucleo-C562RE

The Cytron FD04A requires independent Pulse Width Modulation (PWM) and Direction (DIR) signals for each of its four motor channels.

Target Port on Cytron FD04A

Locate the standard logic pin header input row on the driver board. The pins are labeled clearly on the silk screen: **PWM1, DIR1, PWM2, DIR2, PWM3, DIR3, PWM4, DIR4, and GND**. [\[1\]](#)

Connections & Locations

Cytron Pin Label [1]	Target Wheel Control	Nucleo Pin Name	Physical Pin Location (ST Morpho / Arduino)	Internal STM32 Assignment
PWM1	Front Left Speed	PC6	CN10 Pin 4 (Arduino D1)	TIM1, Channel 1
DIR1	Front Left Direction	PB0	CN7 Pin 34 (Arduino A3)	Standard General GPIO Output
PWM2	Front Right Speed	PC7	CN10 Pin 19 (Arduino D2)	TIM1, Channel 2
DIR2	Front Right Direction	PB1	CN7 Pin 24 (Arduino A4)	Standard General GPIO Output
PWM3	Rear Left Speed	PC8	CN10 Pin 2 (Arduino D3)	TIM1, Channel 3
DIR3	Rear Left Direction	PB2	CN10 Pin 22 (Arduino A5)	Standard General GPIO Output
PWM4	Rear Right Speed	PC9	CN10 Pin 1 (Arduino D4)	TIM1, Channel 4
DIR4	Rear Right Direction	PB12	CN10 Pin 16 (Arduino D5)	Standard General GPIO Output
GND	Logic Common Reference	GND	CN7 Pin 8	Unified Common Ground Rail

Module Warnings & Precautions

- **Timer Synchronization:** Assigning PC6 through PC9 links all four motor channels directly to a single internal hardware timer (**TIM1**). This ensures that the PWM cycles are fully synchronized across all H-bridges, which prevents motor jitter during Mecanum vectoring adjustments.
- **Logic Voltage Range:** The Cytron FD04A is fully compatible with the 3.3V logic outputs from the STM32 Nucleo board, removing the need for external logic level shifters.

Module 3: 4x JG37-520 Encoder Feedback to STM32 Nucleo-C562RE

To maintain precise control during omnidirectional movements, the STM32 uses dedicated hardware timers to track the pulse signals from each motor encoder.

Target Port on Motors

Each JG37-520 motor uses a 6-pin JST-XH cable termination base. The connections must map precisely to avoid damaging the internal Hall-effect sensors.

Connections & Locations

Motor 1: Front Left (FL)

- **Encoder Phase A** (Pin 3) \(\rightarrow\) **PA0** \(\rightarrow\) **CN7 Pin 28** (TIM5_CH1)
- **Encoder Phase B** (Pin 4) \(\rightarrow\) **PA1** \(\rightarrow\) **CN7 Pin 30** (TIM5_CH2)
- **Encoder GND** (Pin 2) \(\rightarrow\) **GND** \(\rightarrow\) **CN7 Pin 22**
- **Encoder VCC** (Pin 5) \(\rightarrow\) **3V3** \(\rightarrow\) **CN6 Pin 4**

Motor 2: Front Right (FR)

- **Encoder Phase A** (Pin 3) \(\rightarrow\) **PA6** \(\rightarrow\) **CN10 Pin 13** (TIM3_CH1)
- **Encoder Phase B** (Pin 4) \(\rightarrow\) **PA7** \(\rightarrow\) **CN10 Pin 14** (TIM3_CH2)
- **Encoder GND** (Pin 2) \(\rightarrow\) **GND** \(\rightarrow\) **CN7 Pin 22**
- **Encoder VCC** (Pin 5) \(\rightarrow\) **3V3** \(\rightarrow\) **CN6 Pin 4**

Motor 3: Rear Left (RL)

- **Encoder Phase A** (Pin 3) \(\rightarrow\) **PB6** \(\rightarrow\) **CN10 Pin 17** (TIM4_CH1)
- **Encoder Phase B** (Pin 4) \(\rightarrow\) **PB7** \(\rightarrow\) **CN10 Pin 19** (TIM4_CH2)
- **Encoder GND** (Pin 2) \(\rightarrow\) **GND** \(\rightarrow\) **CN7 Pin 22**
- **Encoder VCC** (Pin 5) \(\rightarrow\) **3V3** \(\rightarrow\) **CN6 Pin 4**

Motor 4: Rear Right (RR)

- **Encoder Phase A** (Pin 3) \(\rightarrow\) **PC0** \(\rightarrow\) **CN7 Pin 32** (TIM8_CH1)
- **Encoder Phase B** (Pin 4) \(\rightarrow\) **PC1** \(\rightarrow\) **CN7 Pin 34** (TIM8_CH2)
- **Encoder GND** (Pin 2) \(\rightarrow\) **GND** \(\rightarrow\) **CN7 Pin 22**
- **Encoder VCC** (Pin 5) \(\rightarrow\) **3V3** \(\rightarrow\) **CN6 Pin 4**

Module Warnings & Precautions

- **Separating the Wiring Harness:** Pins 1 (**M1**) and 6 (**M2**) on the motor connector carry high-current PWM signals that generate electromagnetic interference (EMI). You must separate **M1** and **M2** from the connector base right away. Twist them together and route them away from the encoder signal lines. Braid the **Encoder GND**, **Phase A**, and **Phase B** wires into a separate, tight triplet to protect them from signal noise.
- **Isolating Ground Paths:** Connect all four **Encoder GND** lines back to a single ground pin on the Nucleo board. Do not ground the encoders through the high-current ground terminals on the Cytron driver. High-current loops can cause ground voltage shifts that lead to dropped or false encoder counts.

Module 4: Cytron FD04A High-Power Output to JG37-520 Motors

This section covers the high-current power lines running from the driver to the motor coils.

Target Ports

- **Cytron Board Output Side:** Heavy screw terminal blocks labeled **M1A/M1B**, **M2A/M2B**, **M3A/M3B**, **M4A/M4B**.
- **Motor Connection Side:** Pin 1 (**M1**) and Pin 6 (**M2**) on the 6-pin connector block.

Connections & Locations

Driver Terminal	Target Motor Unit	Motor Connector Pin Assignment	Expected Action Vector
M1A / M1B	Motor 1 (Front Left)	Pin 1 (M1) / Pin 6 (M2)	Controls the front-left translation coordinate
M2A / M2B	Motor 2 (Front Right)	Pin 1 (M1) / Pin 6 (M2)	Controls the front-right translation coordinate
M3A / M3B	Motor 3 (Rear Left)	Pin 1 (M1) / Pin 6 (M2)	Controls the rear-left translation coordinate
M4A / M4B	Motor 4 (Rear Right)	Pin 1 (M1) / Pin 6 (M2)	Controls the rear-right translation coordinate

Module Warnings & Precautions

- **Wire Gauge and Routing:** Use high-quality multi-strand wire (minimum 18 AWG) for the **M1/M2** lines to handle current spikes when the motor shifts direction. Route these cables along the bottom plate of the chassis, completely separate from the encoder logic lines.

- **Perpendicular Layout Rule:** Where power lines must cross paths with encoder or receiver data lines, arrange them to intersect at a **90-degree angle**. Running power and signal lines parallel to each other creates an inductive coupling path that can introduce noise into your sensor data.
-

Module 5: Jetson Orin Nano Super Brain Preparation

This section outlines the pin layout required to connect the NVIDIA Jetson Orin Nano Super expansion kit to your system. [\[1\]](#)

Target Port on Jetson

Locate the **40-Pin Expansion Header** on the Jetson Orin board. It will use a secondary, independent high-speed UART port on the Nucleo (**USART2**). [\[1, 2\]](#)

Connections & Locations

Jetson 40-Pin Header [1, 2]	Signal Function	Target Nucleo Pin Name	Physical Pin Location (ST Morpho CN7)
Pin 6 (GND)	Shared Ground Reference	GND	CN7 Pin 12
Pin 8 (UART 2 TXD)	Serial Packet Input	PA3 (RX2)	CN7 Pin 37
Pin 10 (UART 2 RXD)	Serial Packet Output	PA2 (TX2)	CN7 Pin 35

Module Warnings & Precautions

- **Preserved Pins:** This setup keeps **USART2** open on pins PA2 and PA3. This ensures you can seamlessly connect the Jetson Orin developer kit in the future without reconfiguring your receiver or encoder inputs.

STM32CubeIDE system configuration profile

- initialize the hardware encoder timers and configure the DMA registers for the CRSF receiver packets

Phase 1: Launching and Creating Your Project

1. Open **STM32CubeIDE**.
 2. Click **File** → **New** → **STM32 Project** from the top menu bar.
 3. A "Target Selection" window will open. Wait for it to finish downloading database updates.
 4. Click on the **Board Selector** tab (located next to the default *Part Selector* tab).
 5. In the **Commercial Part Number** search field, type exactly: NUCLEO-C562RE.
 6. Select the board from the filtered list below, then click **Next** in the bottom right corner.
 7. In the **Project Name** field, type: Mecanum_Rover_Controller.
 8. Ensure **Target Language** is set to **C**, and **Target Binary Type** is set to **Executable**.
 9. Click **Finish**.
 10. A pop-up prompt will ask: *"Initialize all peripherals with their default Mode?"* Click **No**. (This ensures a clean configuration without unwanted pin clashes).
 11. Another prompt will ask: *"Open this perspective associated with the STM32CubeMX graphical configuration tool?"* Click **Yes**.
 12. You will now see a graphical interface showing the microcontroller chip layout with various pin labels. This file is your configuration workspace (Mecanum_Rover_Controller.ioc).
-

Phase 2: System and Clock Configuration

Before configuring timers, the system clock must be running at maximum performance to ensure stable encoder counting and serial processing.

1. On the left-hand column menu, look under the **Categories** list and expand **System Core**.
2. Click on **SYS**.
3. In the center pane under **SYS Mode and Configuration**, locate the **Debug** dropdown field. Change it from *No Debug* to **Serial Wire**. (This is critical; if omitted, your debugger lock-out resets after the first code flash).
4. Now, look at the top horizontal menu tabs of the center screen workspace and click on **Clock Configuration**.
5. Locate the text entry box labeled **HCLK (MHz)** near the center right of the interactive diagram.
6. Delete the existing number, type exactly 144, and hit **Enter**.
7. A prompt will appear stating *"No solver found for this frequency, do you want the tool to run automatic solver?"* Click **OK**. The layout tool will automatically

configure the internal phase-locked loops (PLL) to lock your core frequency to exactly 144 MHz.

Phase 3: Configuring the 4 Hardware Encoder Timers

You will now configure four physical hardware timers into independent, hardware-driven quadrature counter systems. This means the STM32 chip handles the math automatically without bogging down your code.

1. Motor 1 (Front Left) - Configuring Timer 5

1. Go back to the **Pinout & Configuration** tab at the top left.
2. In the left-hand column under **Categories**, click and expand the **Timers** group.
3. Click on **TIM5**.
4. In the top-center box labeled **TIM5 Mode and Configuration**, locate the **Combined Channels** dropdown option. Change it from *Disable* to **Encoder Mode**.
5. Directly underneath, look at the **Configuration** menu panel and select the **Parameter Settings** tab. Modify the options exactly as listed here:
 - **Counter Period (AutoReload Register - ARR):** Change from 65535 to 65535 (or verify it matches this exact default threshold maximum).
 - **Encoder Mode:** Change from *Encoder Mode TI1* to **Encoder Mode TI1 and TI2**. (This tracks both rising and falling signal edges on Phase A and Phase B, quadrupling your encoder resolution accuracy).
6. Look at your chip diagram on the right side of the screen. Verify that pin **PA0** has changed color to green and displays TIM5_CH1, and pin **PA1** displays TIM5_CH2.

2. Motor 2 (Front Right) - Configuring Timer 3

1. In the left-hand column under **Timers**, click on **TIM3**.
2. In the **TIM3 Mode and Configuration** window, look at the **Combined Channels** dropdown option. Select **Encoder Mode**.
3. Move down to the **Configuration** panel and select the **Parameter Settings** tab:
 - **Counter Period (ARR):** Ensure it is set to 65535.
 - **Encoder Mode:** Change from *Encoder Mode TI1* to **Encoder Mode TI1 and TI2**.
4. Look at the chip layout diagram on the right. Verify that pin **PA6** is now labeled TIM3_CH1 and pin **PA7** is labeled TIM3_CH2.

3. Motor 3 (Rear Left) - Configuring Timer 4

1. In the left-hand column under **Timers**, click on **TIM4**.
2. In the **TIM4 Mode and Configuration** window, look at the **Combined Channels** dropdown option. Select **Encoder Mode**.

3. Move down to the **Configuration** panel and select the **Parameter Settings** tab:
 - **Counter Period (ARR):** Ensure it is set to 65535.
 - **Encoder Mode:** Change from *Encoder Mode TI1* to **Encoder Mode TI1 and TI2**.
4. Look at the chip layout diagram on the right. Verify that pin **PB6** is now labeled TIM4_CH1 and pin **PB7** is labeled TIM4_CH2.

4. Motor 4 (Rear Right) - Configuring Timer 8

1. In the left-hand column under **Timers**, click on **TIM8**.
 2. In the **TIM8 Mode and Configuration** window, look at the **Combined Channels** dropdown option. Select **Encoder Mode**.
 3. Move down to the **Configuration** panel and select the **Parameter Settings** tab:
 - **Counter Period (ARR):** Ensure it is set to 65535.
 - **Encoder Mode:** Change from *Encoder Mode TI1* to **Encoder Mode TI1 and TI2**.
 4. Look at the chip layout diagram on the right. Verify that pin **PC0** is now labeled TIM8_CH1 and pin **PC1** is labeled TIM8_CH2.
-

Phase 4: Configuring UART1 + DMA for the RadioMaster ER8

Because ExpressLRS data packets stream constantly at a high speed of 420,000 bps, you must configure **Direct Memory Access (DMA)**. This dumps the data straight into RAM without stressing the CPU.

1. In the left-hand column under **Categories**, scroll down and expand the **Connectivity** group.
2. Click on **USART1**.
3. In the top-center window labeled **USART1 Mode and Configuration**, change the **Mode** dropdown selection from *Disable* to **Asynchronous**.
4. Move down to the **Configuration** panel at the bottom and select the **Parameter Settings** tab:
 - **Baud Rate:** Clear the default value and type exactly 420000.
 - **Word Length:** Ensure it is set to **8 Bits (including Parity)**.
 - **Parity:** Ensure it is set to **None**.
 - **Stop Bits:** Ensure it is set to **1**.
5. Now, select the **DMA Settings** tab located right next to Parameter Settings.
6. Click the blue **Add** button.
7. A new row entry will appear. Click inside its **Select** dropdown box and select **USART1_RX**.
8. Look closely at the properties area at the bottom of the DMA table rows for this channel and adjust these parameters:
 - **Direction:** Ensure it reads *Peripheral To Memory*.

- **Priority:** Change from *Low* to **High**.
 - **Mode:** Change from *Normal* to **Circular**. (This tells the incoming serial data stream to continuously loop over its allocated memory boundary buffer without throwing buffer overflow errors).
 - **Increment Address:** Ensure the **Memory** checkbox is **Checked**, and the **Peripheral** checkbox is **Unchecked**.
 - **Data Width:** Ensure both *Peripheral* and *Memory* fields are set to **BYTE**.
9. Now, select the **NVIC Settings** tab (located next to the DMA Settings tab).
 10. Find the row labeled **USART1 global interrupt**. Look at the column checkbox labeled **Enable** and click to **Check** it. (This handles synchronization markers when packet parsing completes).
 11. Look at the chip layout diagram on the right. Verify that pin **PA9** is now green and labeled USART1_TX, and pin **PA10** is green and labeled USART1_RX.
-

Phase 5: Generating the Driver Code Framework

1. Look at the top icon menu bar of STM32CubeIDE and locate the **Save** icon (the single floppy disk icon) or press Ctrl + S.
 2. A pop-up prompt will window will ask: "*Do you want to generate Code?*" Click **Yes**.
 3. If an additional prompt asks "*Associated perspective transformation application?*", click **Yes** to shift into the C/C++ perspective layout view.
 4. The background code generation scripts will now run. On the left side of your screen inside the **Project Explorer** tab tree, expand your folder paths to: Mecanum_Rover_Controller → Core → Src → and double-click to open main.c.
-

Phase 6: Injecting Runtime Activation Firmware

Scroll down inside your newly generated main.c file. You will see several commented blocks that look like `/* USER CODE BEGIN ... */` and `/* USER CODE END ... */`. Only write code within these specific boundaries, or STM32CubeIDE will delete your code next time you make changes in the graphical workspace.

1. Allocate the Global Serial Receiver Memory Buffer

Locate `/* USER CODE BEGIN PV */` (Private Variables). Paste the following tracking buffers right below it:

```
```bash
/* USER CODE BEGIN PV */
#define CRSF_BUFFER_SIZE 64
uint8_t crsf_rx_buffer[CRSF_BUFFER_SIZE]; // DMA directly streams raw bytes here
```

```
int16_t encoder_count_motor1 = 0;
int16_t encoder_count_motor2 = 0;
int16_t encoder_count_motor3 = 0;
int16_t encoder_count_motor4 = 0;
/* USER CODE END PV */
```

```

2. Start the Peripherals Before Entering the System Loop

Scroll down further until you find the int main(void) execution setup entry point. Locate /* USER CODE BEGIN 2 */. Paste these exact function calls here to activate the timer blocks and link the DMA data stream to the serial interface:

```
```bash
/* USER CODE BEGIN 2 */
// Turn on hardware quadrature processing loops for all 4 encoder sets
HAL_TIM_Encoder_Start(&htim3, TIM_CHANNEL_ALL);
HAL_TIM_Encoder_Start(&htim4, TIM_CHANNEL_ALL);
HAL_TIM_Encoder_Start(&htim5, TIM_CHANNEL_ALL);
HAL_TIM_Encoder_Start(&htim8, TIM_CHANNEL_ALL);

// Launch background DMA streaming loop for incoming ExpressLRS data packets
HAL_UART_Receive_DMA(&huart1, crsf_rx_buffer, CRSF_BUFFER_SIZE);
/* USER CODE END 2 */
```

```

3. Continuous Tracking Loop Data Acquisition

Scroll down to your infinite execution engine loop while (1). Locate /* USER CODE BEGIN 3 */. Paste these query functions inside the loop to continuously pull values out of the internal hardware tracking registers:

```
```bash
/* USER CODE BEGIN 3 */
// Pull current raw positions directly out of internal tracking registers
encoder_count_motor1 = (int16_t)__HAL_TIM_GET_COUNTER(&htim5); // Front Left
encoder_count_motor2 = (int16_t)__HAL_TIM_GET_COUNTER(&htim3); // Front Right
encoder_count_motor3 = (int16_t)__HAL_TIM_GET_COUNTER(&htim4); // Rear Left
encoder_count_motor4 = (int16_t)__HAL_TIM_GET_COUNTER(&htim8); // Rear Right

// Add your kinematics math mixing code right below here
```

```

```
    HAL_Delay(10); // Maintain a stable calculation cycle loop (100Hz frequency)
}
/* USER CODE END 3 */
` ``
```

4. Compile and Build

1. Connect your STM32 Nucleo C562RE to your PC via a USB cable.
2. In the top toolbar, click the **Build** icon (the hammer emblem) or press Ctrl + B.
Verify in the bottom console window that the build finishes with **0 errors**.
3. Click the **Run** icon (the green play button arrow) to upload the firmware profile configuration straight into the target MCU flash memory layout.